Page Pulse - Architecture Document

Executive Summary

Page Pulse is a production-grade URL audit service designed to handle 10,000+ audits per day with configurable caching, rate limiting, and structured observability. This document outlines the system architecture, technology decisions, failure modes, and scaling strategy.

System Architecture

High-Level Components

Client Layer

- React 19 + Tailwind CSS + shadcn/ui
- Audit Dashboard (URL input, results display)
- History Table (past audits)
- Architecture Documentation Page

API Layer

- Express.js + tRPC 11
- Request routing and middleware
- Authentication (Manus OAuth)
- Request ID generation and propagation
- Error handling and structured responses

Business Logic Layer

- **Audit Service**: URL validation, HTTP fetch with timeout, metadata extraction, redirect chain tracking, semaphore-based concurrency limiting
- **Cache Layer**: In-memory cache with TTL, cache key: audit:${url}, configurable TTL (default: 5 minutes), cache hit/miss tracking
- **Rate Limiter**: Token bucket algorithm per IP, configurable request limits (default: 100/min), separate bucket per unique IP address

- **Structured Logger**: JSON-formatted logs with requestId, log levels: debug, info, warn, error, contextual data (URL, response time, status)

Data Persistence Layer

- MySQL Database (via Drizzle ORM)
- users table (authentication)
- audits table (audit history)
- Indexes on userId, url, createdAt

Data Flow

Request Processing Pipeline

1. **HTTP Request arrives** at Express server
2. **Context middleware** creates requestId (req_*)
3. **Set X-Request-ID** response header
4. **tRPC procedure invoked** (audit.performAudit)
5. **Extract client IP** from x-forwarded-for or socket.remoteAddress
6. **Rate limiter checks** if IP has available tokens
   - If limit exceeded → Return 429 with Retry-After header
   - If allowed → Continue
7. **Generate cache key** from URL
8. **Check in-memory cache**
   - Cache HIT → Return cached result with X-Cache: HIT header
   - Cache MISS → Continue
9. **Validate URL format** (protocol, length, format)
   - If invalid → Return structured error
   - If valid → Continue
10. **Acquire semaphore permit** (concurrency control)
    - If available → Continue
    - If unavailable → Queue and wait

11. **Fetch URL** with AbortController timeout
    - Success → Extract metadata, headers, status
    - Timeout → Return error with timeout message
    - Network error → Return error with details
12. **Release semaphore permit**
13. **Cache result** (if no error)
14. **Save to database** (if user authenticated)
15. **Return result** with X-Request-ID and X-Cache: MISS headers

---

Queueing Strategy

Current Architecture (Single Instance)

Semaphore-Based Queueing:

- Semaphore with configurable permits (default: 10)
- Excess requests queue in JavaScript Promise array
- FIFO processing order
- No persistence across restarts

Advantages:

- Simple, zero-configuration
- Low latency for queued requests
- No external dependencies

Limitations:

- Queued requests lost on server restart
- Not suitable for multi-instance deployments
- Memory-bounded by available heap

Scaled Architecture (10,000 Audits/Day)

Message Queue-Based Queueing:

```
Client Request → API Server → Message Queue (RabbitMQ/Kafka)
                                      ↓
                                Worker Pool (N workers)
                                      ↓
                                Result Cache (Redis)
```

Benefits:

- Decouples API from audit processing
- Survives server restarts
- Scales horizontally with worker count
- Enables priority queuing
- Dead letter queue for failed audits

Implementation:

- API server enqueues audit requests immediately
- Returns 202 Accepted with requestId
- Client polls for results or uses WebSocket
- Workers process queue concurrently
- Results cached in Redis for all instances

Technology Decisions

1. tRPC over REST API

Chosen: tRPC

- End-to-end type safety (TypeScript)
- Automatic client generation
- Smaller payload sizes with superjson
- Built-in error handling

Rejected: REST

- Manual schema validation required
- Separate client generation step
- Larger JSON payloads
- Error handling less consistent

2. In-Memory Cache vs Redis

Chosen: In-Memory Cache (Single Instance)

- Zero operational overhead
- Sub-millisecond latency
- No network round-trip
- Suitable for single-instance deployments

Rejected: Redis

- Adds operational complexity
- Network latency (1-5ms)
- Requires Redis infrastructure
- Necessary only for multi-instance setups

For 10,000 Audits/Day: Migrate to Redis

- Enables cache sharing across instances
- Supports cache invalidation across workers
- Enables pub/sub for real-time updates

3. Token Bucket Rate Limiter

Chosen: Token Bucket

- Smooth rate limiting with burst tolerance
- Per-IP isolation prevents cascading failures
- Configurable refill rate
- Simple implementation

Rejected: Fixed Window Counter

- Hard cutoffs at window boundaries
- Unfair to requests at window edges
- No burst tolerance

4. Semaphore for Concurrency Control

Chosen: Semaphore Pattern

- Bounded concurrent requests
- Prevents resource exhaustion
- Simple queue management
- Configurable permit count

Rejected: Unbounded Concurrency

- Risk of memory exhaustion
- Uncontrolled resource usage
- Poor performance under load

5. Structured JSON Logging

Chosen: Structured JSON Logs

- Machine-parseable format
- Unique requestId for correlation
- Contextual data included
- Integrates with log aggregation tools

Rejected: Unstructured Text Logs

- Hard to parse and correlate
- Difficult to aggregate and analyze
- Poor for debugging distributed systems

6. MySQL with Drizzle ORM

Chosen: MySQL + Drizzle

- Type-safe database queries

- Automatic migration generation
- Excellent TypeScript support
- Lightweight ORM

Rejected: MongoDB

- Overkill for structured audit data
- Harder to enforce schema consistency
- Less suitable for relational data

Failure Modes & Mitigations

1. Target URL Timeout or Unreachable

Failure Mode:

- URL takes >10 seconds to respond
- Server is offline or unreachable
- Network connectivity issues

Mitigation:

- Configurable timeout (default: 10s) with AbortController
- Returns structured error response with requestId
- User can retry or try different URL
- Timeout error logged with context

Monitoring:

- Track timeout rate per domain
- Alert if timeout rate exceeds threshold
- Log timeout errors with URL for analysis

2. Rate Limit Exhaustion

Failure Mode:

- Client makes >100 requests per minute

- Malicious actor attempts DDoS
- Legitimate spike in traffic

Mitigation:

- Token bucket refills over time
- Returns 429 with Retry-After header
- Configurable limits per environment
- Per-IP isolation prevents cascading failures

Monitoring:

- Track rate limit rejections per IP
- Alert on unusual patterns
- Log rejected requests with IP and timestamp

3. Memory Exhaustion (Cache)

Failure Mode:

- Cache grows unbounded
- Server runs out of heap memory
- OOM killer terminates process

Mitigation:

- Configurable TTL ensures entries expire (default: 5 min)
- In-memory cache is bounded by TTL
- Automatic cleanup of expired entries
- For production scale: use Redis with eviction policies

Monitoring:

- Track cache size and hit/miss ratio
- Alert if cache size exceeds threshold
- Monitor heap usage

4. Database Connection Failure

Failure Mode:

- MySQL connection drops
- Database becomes unavailable
- Network partition

Mitigation:

- Audit service works without database (public audits)
- Audit history only saved for authenticated users
- Graceful degradation with error logging
- Connection pooling with retry logic

Monitoring:

- Track database connection errors
- Alert on connection pool exhaustion
- Monitor query latency

5. Concurrent Request Spike

Failure Mode:

- Sudden traffic spike (500+ concurrent requests)
- Semaphore permits exhausted
- Requests queue indefinitely

Mitigation:

- Semaphore limits concurrent fetches (default: 10)
- Excess requests queue with FIFO ordering
- Configurable concurrency limit per environment
- For production: use message queue with worker pool

Monitoring:

- Track semaphore queue depth

- Alert if queue grows too large
- Monitor request latency percentiles

Observability & Monitoring

Request Tracing

Request ID Propagation:

- Every request gets unique `requestId` (format: `req_*`)
- Included in `X-Request-ID` response header
- Appears in all structured logs
- Enables end-to-end tracing across components

Example:

```
Request: audit.performAudit({ url: "https://example.com" })
Response Header: X-Request-ID: req_abc123xyz
Log Entry: { requestId: "req_abc123xyz", message: "Audit completed", ...
```

Structured Logging

Log Format:

```
{
  "requestId": "req_abc123xyz",
  "timestamp": "2026-07-25T05:50:00.000Z",
  "level": "info",
  "message": "Audit completed",
  "data": {
    "url": "https://example.com",
    "responseTime": 245,
    "statusCode": 200,
    "cached": false
  }
}
```

Log Levels:

- `debug` : Detailed diagnostic information
- `info` : General informational messages
- `warn` : Warning conditions
- `error` : Error conditions

Metrics Endpoints

Cache Statistics:

```
GET /trpc/audit.getCacheStats
Response: { size: 42, ttlMs: 300000 }
```

Rate Limiter Statistics:

```
GET /trpc/audit.getRateLimiterStats
Response: { maxTokens: 100, refillRatePerSecond: 1.67, activeBuckets: 23 }
```

Response Headers:

- `X-Request-ID` : Unique request identifier
- `X-Cache` : “HIT” or “MISS”
- `Retry-After` : Seconds to wait (if rate limited)

Monitoring Dashboard

Key Metrics:

- Request rate (requests/second)
- Cache hit ratio (%)
- Rate limit rejection rate (%)
- Average response time (ms)
- Error rate (%)
- Semaphore queue depth

- Database connection pool usage

Alerts:

- Cache hit ratio < 30%
- Rate limit rejection rate > 5%
- Average response time > 2000ms
- Error rate > 1%
- Semaphore queue depth > 100
- Database connection errors

Rollback Strategy

Git-Based Versioning:

- Every commit is a potential rollback point
- GitHub Actions CI verifies all commits
- Checkpoints capture working states

Database Migrations:

- All migrations are reversible
- Rollback script generated for each migration
- Test rollbacks in staging before production

Deployment Process:

1. Create checkpoint before deploy
2. Run tests and verify build
3. Deploy to production
4. Monitor metrics for 5 minutes
5. If issues detected, rollback to previous checkpoint

Rollback Execution:

```
# Identify bad commit
git log --oneline

# Rollback to previous commit
git reset --hard <commit-hash>

# Reverse database migration if needed
pnpm db:rollback

# Restart server
npm start
```

Scaling to 10,000 Audits/Day

Current Capacity

Single Instance:

- Concurrency limit: 10 (semaphore)
- Request timeout: 10 seconds
- Cache TTL: 5 minutes
- Rate limit: 100 requests/minute per IP

Estimated Capacity:

- ~1,000 audits/day (assuming 10 second average response time)
- Peak: ~100 concurrent requests

Scaling Strategy

Phase 1: Optimize Single Instance (1,000 - 5,000 audits/day)

- Increase semaphore permits to 50
- Reduce cache TTL to 2 minutes
- Increase rate limit to 500 requests/minute
- Add database read replicas

- Monitor and tune based on metrics

Phase 2: Horizontal Scaling (5,000 - 20,000 audits/day)

- Deploy 3-5 instances behind load balancer
- Replace in-memory cache with Redis
- Implement distributed rate limiting
- Add message queue for async processing
- Database connection pooling across instances

Phase 3: Microservices (20,000+ audits/day)

- Separate API server from audit workers
- Dedicated worker pool (10-20 workers)
- Message queue with priority support
- Distributed cache (Redis cluster)
- Database sharding by user ID
- CDN for static assets

Infrastructure Requirements

Phase 1 (Single Instance):

- 1x Node.js server (2 vCPU, 4GB RAM)
- 1x MySQL server (2 vCPU, 8GB RAM)
- Load balancer (optional)

Phase 2 (Multi-Instance):

- 3-5x Node.js servers (2 vCPU, 4GB RAM each)
- 1x Redis server (2 vCPU, 4GB RAM)
- 1x MySQL server (4 vCPU, 16GB RAM)
- 1x RabbitMQ server (2 vCPU, 4GB RAM)
- Load balancer with health checks

Phase 3 (Microservices):

- 5x API servers (2 vCPU, 4GB RAM each)

- 10x Worker servers (2 vCPU, 2GB RAM each)
- 1x Redis cluster (3 nodes, 4GB each)
- 1x MySQL cluster (3 nodes, 16GB each)
- 1x RabbitMQ cluster (3 nodes, 4GB each)
- Load balancer with auto-scaling

Conclusion

Page Pulse is architected for production use with comprehensive error handling, observability, and scaling capabilities. The current single-instance design is suitable for up to 5,000 audits/day, with clear migration paths to handle 10,000+ audits/day through horizontal scaling and distributed architecture.

Key Strengths:

- Type-safe end-to-end with TypeScript and tRPC
- Structured observability with request IDs and JSON logging
- Configurable caching and rate limiting
- Graceful degradation under failure
- Clear scaling strategy with proven patterns

Built for Digital Heroes Training Task

Visit digitalheroesco.com for more information.